

# Sistema embebido de monitoreo de calidad del aire con sensores MQ-135 y SDS011, aplicación web y alertas preventivas.

Hernán Villafuerte<sup>1</sup>, Emerson Solano<sup>2</sup>, Clara Quispe<sup>2</sup>, Dafne Solórzano<sup>2</sup>, Astrid Quezada<sup>2</sup>

<sup>1</sup> Universidad Nacional Mayor de San Marcos (UNMSM), Lima, Perú

<sup>2</sup> Facultad de Ingeniería Industrial y de Sistemas, Universidad Nacional Federico Villarreal (UNFV), Lima, Perú

## Resumen

*La contaminación del aire en zonas urbanas de Lima constituye un problema de salud pública que ha ido agravándose con el crecimiento vehicular e industrial. Frente a esto, se desarrolló un sistema embebido de monitoreo ambiental basado en el microcontrolador ESP32, integrando los sensores MQ-135 para detección de gases y SDS011 para material particulado. El sistema emplea una máquina de estados finitos que clasifica la calidad del aire en tres niveles (Bueno, Moderado y Dañino) permitiendo una respuesta automatizada ante variaciones en los parámetros medidos. Los datos recopilados se envían a una aplicación web desarrollada con Flask y SQLite, que ofrece visualización en tiempo real, un módulo de gestión de alertas y un asistente virtual con capacidad de respuesta contextualizada mediante IA, accesible desde cualquier dispositivo conectado a la red.*

*El prototipo fue construido e implementado físicamente, y su desempeño se evaluó mediante cuatro métricas cuantitativas: tiempo de respuesta de alerta, tiempo de actualización de datos, sensibilidad de detección de partículas y nivel de detección de gases. Los resultados mostraron un uso de memoria de 71,0% en Flash y 14,4% en RAM, lo que evidencia una utilización eficiente de los recursos del microcontrolador. Este trabajo sienta las bases para escalar la solución hacia redes de monitoreo distribuidas dentro del paradigma IoT e incorporar modelos de Inteligencia Artificial que permitan anticipar episodios críticos de contaminación.*

## Palabras clave

*ESP32; calidad del aire; máquina de estados finitos; PM2.5; PM10; MQ-135; SDS011; aplicación web; IoT.*

## I. DESCRIPCIÓN DEL PROBLEMA Y CONTEXTO

La contaminación del aire es un problema ambiental y de salud pública que afecta principalmente a las zonas urbanas, donde el tránsito vehicular, las actividades industriales y la presencia de partículas suspendidas alteran la calidad del ambiente. Dentro de estos contaminantes, el material particulado PM10 y PM2.5 es de especial importancia, debido a que puede permanecer suspendido en el aire y ser inhalado por las personas. En el caso del PM2.5, su menor tamaño permite que ingrese con mayor

profundidad al sistema respiratorio, por lo que su monitoreo resulta necesario para identificar condiciones que puedan representar un riesgo para la salud.

En el contexto peruano, la calidad del aire aún presenta desafíos importantes. Diversos estudios señalan que los valores establecidos en el Perú para algunos contaminantes no se ajustan completamente a las recomendaciones de la Organización Mundial de la Salud (OMS) [1]. Por ejemplo, para PM10 la OMS recomienda un valor diario de 45  $\mu\text{g}/\text{m}^3$ , mientras que el estándar peruano considera 100  $\mu\text{g}/\text{m}^3$ ; para PM2.5, la OMS recomienda 15  $\mu\text{g}/\text{m}^3$  como promedio diario frente a los 50  $\mu\text{g}/\text{m}^3$  establecidos en el Perú. Cabe señalar que, en Estados Unidos, la Agencia de Protección Ambiental (EPA) define para PM2.5 una escala por tramos (0-12, 12.1-35.4, 35.5-55.4  $\mu\text{g}/\text{m}^3$ , entre otros) utilizada para construir el Índice de Calidad del Aire (AQI) [4]. Esta diferencia entre estándares evidencia la necesidad de contar con alternativas de monitoreo accesibles que permitan reconocer condiciones ambientales desfavorables de manera preventiva, independientemente del estándar de referencia adoptado.

En Lima Metropolitana y el Callao, el material particulado PM10 y PM2.5 ha sido uno de los principales indicadores del deterioro de la calidad del aire. Un análisis de datos de estaciones de monitoreo de DIGESA y SENAMHI durante el periodo 2001-2014 encontró que los promedios anuales de PM10 y PM2.5 superaron significativamente los estándares nacionales y las guías de la OMS en varias estaciones, identificando además que las zonas norte, sur y este de Lima presentaron los mayores valores de concentración de material particulado [2].

El impacto de este problema no se limita al aspecto ambiental, sino también a la salud de la población. Un estudio que evaluó la exposición diaria a PM2.5 y su relación con la mortalidad cardiorrespiratoria en Lima entre 2010 y 2016, analizando 86 970 muertes por enfermedades respiratorias y cardiovasculares, encontró una asociación significativa entre el incremento de PM2.5 y la mortalidad cardiorrespiratoria, especialmente en personas mayores de 65 años [3]. Estos antecedentes muestran que el monitoreo del material particulado puede aportar

información útil para la prevención, la sensibilización ambiental y la toma de decisiones. Frente a esta problemática, se desarrolló un sistema embebido de monitoreo de calidad del aire basado en ESP32, integrando los sensores MQ-135 y SDS011, junto con una aplicación web que permite visualizar las mediciones en tiempo real, generar alertas preventivas, gestionar los umbrales de clasificación e interactuar con un asistente virtual que responde consultas sobre la calidad del aire con base en los datos del sensor. El sensor MQ-135 permite detectar variaciones asociadas a humo o gases contaminantes como indicador general de deterioro de la calidad del aire, mientras que el sensor SDS011 permite medir concentraciones de partículas PM2.5 y PM10 mediante tecnología de dispersión láser. Este sistema beneficia principalmente a personas que viven, estudian o trabajan en zonas urbanas con posible exposición a contaminantes, así como a adultos mayores, niños y personas con antecedentes respiratorios.

## II. OBJETIVOS

### A. Objetivo General

Diseñar e implementar un sistema embebido de monitoreo de calidad del aire basado en ESP32 con los sensores MQ-135 y SDS011, capaz de detectar gases contaminantes y partículas PM2.5/PM10, clasificar el estado ambiental mediante una máquina de estados finitos, visualizar los datos a través de una aplicación web en tiempo real y emitir alertas preventivas ante condiciones que representen un riesgo para la salud.

### B. Objetivos Específicos

- Medir la concentración de partículas PM2.5 y PM10 mediante el sensor SDS011, e incorporar el sensor MQ-135 para detectar variaciones asociadas a gases contaminantes.
- Clasificar la calidad del aire en niveles de alerta (Bueno, Moderado, Dañino) mediante una máquina de estados finitos implementada en el firmware.
- Desarrollar una aplicación web conectada por WiFi al ESP32 para visualizar los datos ambientales en tiempo real, con gestión de usuarios y umbrales configurables y un asistente virtual integrado con soporte de inteligencia artificial para consultas contextuales.
- Implementar alertas preventivas e información interpretativa sobre posibles efectos en la salud, mediante indicadores LED, buzzer y la interfaz web.
- Determinar el nivel de variación de los datos ambientales obtenidos por los sensores en diferentes condiciones de prueba, mediante métricas cuantitativas.

## III. DISEÑO DEL SISTEMA

La arquitectura propuesta se organiza en dos subsistemas que se comunican mediante HTTP a través de una red WiFi local. El primero corresponde a un subsistema embebido basado en ESP32, responsable de la adquisición de datos de los sensores, la clasificación de la calidad del aire mediante una máquina de estados finitos (MEF) y la activación de los actuadores de alerta. El segundo corresponde a una aplicación web desarrollada en Python con Flask, encargada del almacenamiento de los registros, la visualización en tiempo real, la gestión de usuarios y roles, y la administración de los umbrales de clasificación.

### A. Componentes de Hardware

La Tabla I resume los componentes de hardware utilizados en el prototipo.

TABLA I. COMPONENTES DE HARDWARE DEL SISTEMA

| Componente                 | Interfaz              | Función                                                                |
|----------------------------|-----------------------|------------------------------------------------------------------------|
| ESP32                      | WiFi, UART, ADC, GPIO | Unidad central de procesamiento, ejecuta la MEF y la comunicación HTTP |
| SDS011                     | UART2 (9600 bps)      | Mide PM2.5 y PM10 por dispersión láser                                 |
| MQ-135                     | ADC1_CH7 (12 bits)    | Detecta gases contaminantes (CO2, humo, NH3, COV)                      |
| LEDs (verde/amarillo/rojo) | GPIO digital          | Indicador visual del estado de la MEF                                  |
| Buzzer activo              | GPIO5 (digital)       | Alarma sonora en estado Crítico                                        |
| Resistencias 220 $\Omega$  | -                     | Protección de corriente en LEDs                                        |

### B. Descripción General del Funcionamiento

El proyecto propuesto consiste en el desarrollo de un sistema embebido para el monitoreo de la calidad del aire utilizando un microcontrolador ESP32, un sensor de partículas SDS011 y un sensor de gases MQ-135. La finalidad del sistema es detectar en tiempo real diferentes contaminantes presentes en el ambiente y advertir al usuario cuando se alcancen niveles que puedan representar un riesgo para la salud. Para ello, el SDS011 realiza la medición de partículas PM2.5 y PM10 mediante tecnología láser, mientras que el MQ-135 complementa el monitoreo detectando gases y compuestos asociados a la contaminación atmosférica. La información generada por ambos sensores es enviada al ESP32, donde se lleva a cabo el procesamiento de los datos y su comparación con umbrales previamente establecidos para determinar el nivel de calidad del aire.

A partir de esta evaluación, el sistema emplea una máquina de estados finitos (FSM) que permite clasificar las condiciones ambientales en estados de monitoreo normal, alerta preventiva

y alerta crítica. Según el estado detectado, se activan diferentes mecanismos de notificación mediante LEDs indicadores y un buzzer, proporcionando una respuesta rápida y comprensible para el usuario.

Complementariamente, se plantea el desarrollo de una aplicación local encargada de visualizar las mediciones obtenidas y almacenar registros históricos para su posterior consulta. Esta funcionalidad amplía las capacidades del sistema al permitir un seguimiento continuo de las condiciones ambientales monitoreadas.

Finalmente, debido a las capacidades de conectividad incorporadas en el ESP32, se considera como una línea de desarrollo futura la integración de servicios IoT para el monitoreo remoto y la gestión centralizada de la información generada.

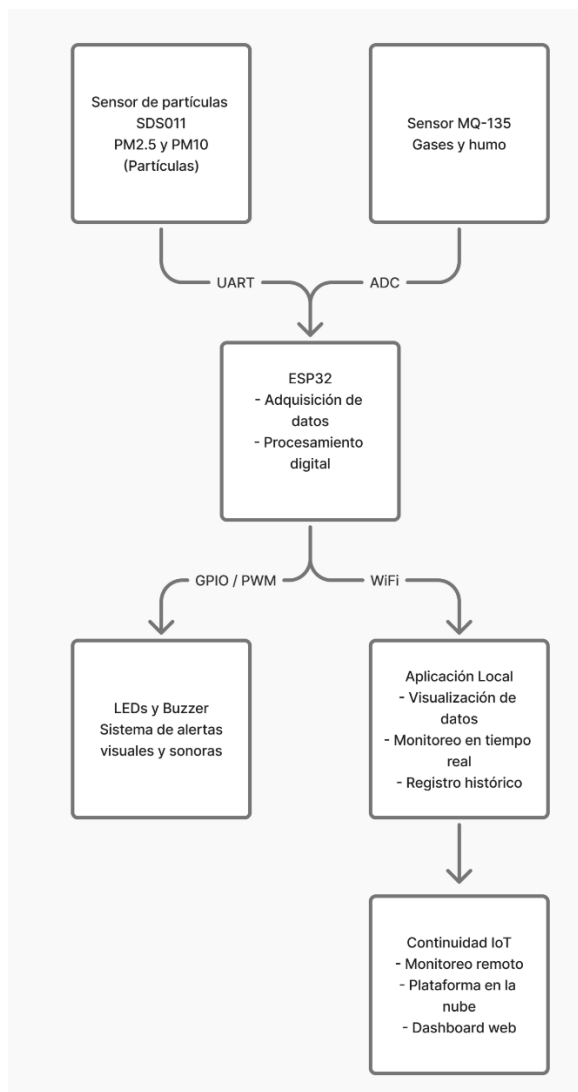


Fig. 1. Diagrama de bloques preliminar del sistema embebido de monitoreo y análisis de calidad del aire basado en ESP32 con sensores SDS011 y MQ-135.

### C. Arquitectura de la Aplicación Web

A diferencia de lo planteado inicialmente como aplicación móvil local, el valor agregado del proyecto se implementó como una aplicación web desarrollada en Python con el microframework Flask, siguiendo el patrón de diseño Modelo-Vista-Controlador (MVC). La capa de Modelo gestiona la persistencia mediante SQLite (tablas usuarios, umbrales e historial) y la lógica de clasificación del aire; la capa de Controlador se organiza en Blueprints independientes (auth, sensor, historial, admin) que exponen las rutas HTTP; y la capa de Vista utiliza plantillas Jinja2 (dashboard, historial, recomendaciones, panel de administración) renderizadas en el servidor, con JavaScript en el cliente encargado del refresco periódico de los datos.

La comunicación entre el firmware y la aplicación web se realiza mediante el protocolo HTTP mediante peticiones REST, mientras que la actualización en la interfaz del usuario se logra mediante sondeo periódico (polling) cada 2000 ms desde el navegador hacia el endpoint /api/estado, en lugar de una conexión persistente por WebSocket. Esta decisión de diseño simplifica la implementación y resulta adecuada para la frecuencia de muestreo del sistema, sin perjuicio de que en una iteración futura el mecanismo de polling pueda sustituirse por una conexión en tiempo real.

El acceso a la aplicación se realiza desde cualquier dispositivo conectado a la misma red WiFi local (PC, tablet o teléfono), a través del navegador, sin requerir instalación de una aplicación nativa.

Como componente adicional de la capa de interacción, se integró un asistente virtual accesible desde el dashboard y la página de recomendaciones. El asistente implementa una arquitectura híbrida: un módulo de reconocimiento de intenciones por palabras clave que cubre las consultas más frecuentes (estado actual, recomendaciones por grupo de riesgo, explicación de contaminantes, historial del día), y un módulo de respaldo basado en la API de Gemini (modelo gemini-2.0-flash) que se activa únicamente cuando el mensaje del usuario no coincide con ninguna intención predefinida. En cada llamada al módulo de IA se incluye como contexto el estado actual del sensor (PM2.5, PM10, MQ-135 y nivel de alerta), lo que permite respuestas contextualizadas a las condiciones ambientales en tiempo real.

## IV. LÓGICA DE CONTROL Y ARQUITECTURA DEL MICROCONTROLADOR

### A. Máquina de Estados Finitos (MEF)

La lógica de control del sistema fue modelada mediante una Máquina de Estados Finitos (MEF), conocida en la literatura como *Finite State Machine (FSM)*. Este modelo permite

representar de manera formal el comportamiento del sistema mediante un conjunto de estados, eventos de entrada, transiciones y salidas asociadas, describiendo la secuencia lógica de operación durante el monitoreo de la calidad del aire. La MEF está conformada por tres estados de operación: Normal (S0), Preventivo (S1) y Crítico (S2). Las transiciones entre estos estados dependen de un evento de entrada generado a partir de la clasificación de las mediciones obtenidas por los sensores MQ-135 y SDS011. Dicho proceso compara las concentraciones registradas con los umbrales establecidos y asigna un único evento que representa la condición global de la calidad del aire. Para garantizar una respuesta segura, el criterio de clasificación adopta siempre el nivel de mayor severidad detectado entre todas las variables monitoreadas. Con el propósito de simplificar la representación formal de la máquina de estados, tanto los eventos de entrada como los estados y las salidas fueron codificados mediante valores binarios. Esta codificación proporciona una representación compacta del comportamiento de la MEF y facilita la interpretación del diagrama de estados y de las tablas de transición.

TABLA II. CODIFICACIÓN DE EVENTOS DE ENTRADA

| Evento | Código | Condición                 |
|--------|--------|---------------------------|
| E0     | 00     | Calidad del aire Buena    |
| E1     | 01     | Calidad del aire Moderada |
| E2     | 10     | Calidad del aire Dañina   |

Los eventos representan el resultado del proceso de clasificación de la calidad del aire y constituyen las entradas de la Máquina de Estados Finitos. Cada evento es generado después de evaluar conjuntamente las mediciones de los sensores. Cuando alguna de las variables monitoreadas alcanza un nivel de mayor severidad, la MEF recibe el evento correspondiente y ejecuta la transición de estado asociada.

TABLA III. TABLA DE CODIFICACIÓN DE ESTADOS

| Estado | Código | Descripción |
|--------|--------|-------------|
| S0     | 00     | Normal      |
| S1     | 01     | Preventivo  |
| S2     | 10     | Crítico     |

La Máquina de Estados Finitos está conformada por tres estados de operación, identificados mediante una codificación binaria de dos bits. Al iniciar la operación del sistema, la MEF comienza su ejecución en el estado S0 (Normal), una vez finalizada la inicialización del ESP32 y de los dispositivos asociados. A partir de ese momento, la ocurrencia de un nuevo evento determina si la máquina permanece en el estado actual o realiza una transición hacia otro estado.

TABLA IV. CODIFICACIÓN DE LAS SALIDAS DE LA MEF

| Estado | Led V. | Led A. | Led R. | B | Salida |
|--------|--------|--------|--------|---|--------|
| S0     | 1      | 0      | 0      | 0 | 1000   |
| S1     | 0      | 1      | 0      | 0 | 0100   |
| S2     | 0      | 0      | 1      | 1 | 0011   |

Cada estado posee una única combinación de salidas asociada, responsable del control de los actuadores del sistema. En esta codificación, el valor lógico 1 indica que el actuador correspondiente se encuentra activado, mientras que el valor lógico 0 indica que permanece desactivado.

Representación de la Máquina de Estados Finitos

La Fig. 2 presenta la representación formal de la Máquina de Estados Finitos utilizada para modelar el comportamiento del sistema. La ejecución comienza en el estado S0 (Normal), indicado mediante la transición de inicio. A partir de este estado, la evolución de la máquina depende únicamente del evento de entrada recibido. Cada transición se representa mediante la notación Entrada/Salida, donde la entrada corresponde al evento generado por el proceso de clasificación y la salida representa el código binario asociado a los actuadores de la MEF. Por ejemplo, la transición 00/1000 indica que, al generarse el evento E0, la máquina permanece o transita al estado S0, activando únicamente el LED verde. La estructura de la MEF permite realizar transiciones entre cualquiera de sus tres estados de operación. En consecuencia, la máquina puede permanecer en el mismo estado cuando la condición ambiental no presenta cambios o transitar directamente entre los estados Normal, Preventivo y Crítico, de acuerdo con el evento generado. Esta característica permite actualizar continuamente el estado del sistema conforme se reciben nuevas mediciones, garantizando una respuesta inmediata frente a las variaciones de la calidad del aire.

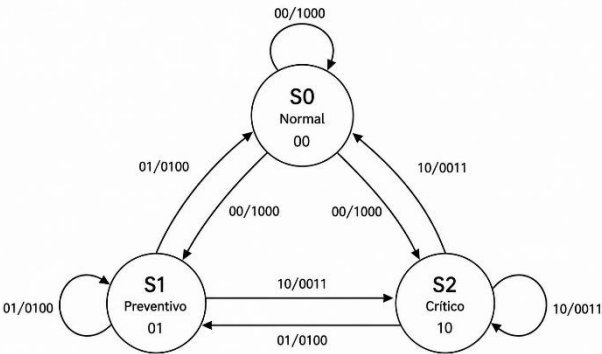


Fig. 2. Diagrama de la máquina de estados finitos implementada en el firmware del ESP32

TABLA V. DE TRANSICIÓN DE ESTADOS

| Estado Actual | 00 (E0) | 01 (E1) | 10 (E2) |
|---------------|---------|---------|---------|
| 00 (S0)       | 00 (S0) | 01 (S1) | 10 (S2) |
| 01 (S1)       | 00 (S0) | 01 (S1) | 10 (S2) |
| 10 (S2)       | 00 (S0) | 01 (S1) | 10 (S2) |

La Tabla V resume el comportamiento de la Máquina de Estados Finitos. Para cada combinación de estado actual y evento de entrada existe una única transición posible, lo que define una MEF determinística. Esta propiedad garantiza un comportamiento predecible y consistente durante toda la operación del sistema, asegurando que el estado de funcionamiento refleje en todo momento la condición de calidad del aire detectada.

### B. Mapa de Memoria del ESP32

La memoria Flash almacena el programa principal del controlador (incluyendo la lectura de los sensores, la MEF, la gestión de WiFi y el intercambio de datos por HTTP) además del bootloader y la tabla de particiones. La memoria RAM almacena las variables temporales: lecturas de sensores, umbrales descargados desde la aplicación web, el estado actual de la MEF y los buffers de comunicación y de procesamiento JSON.

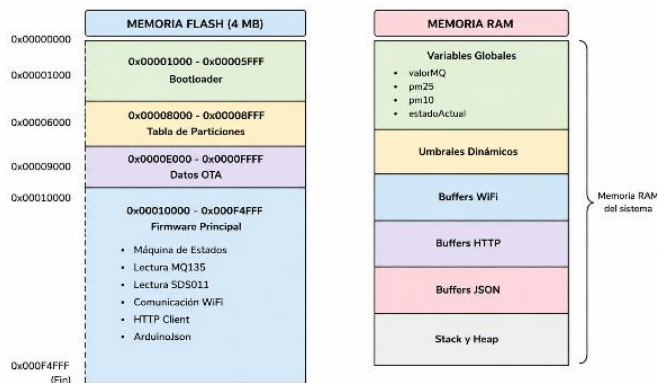


Fig. 3. Mapa de memoria simplificado del ESP32 aplicado al sistema de monitoreo de calidad del aire.

TABLA V. ORGANIZACIÓN DE LA MEMORIA FLASH DEL ESP32

| Dir. Inicial | Dir. Final | Tamaño    | Descripción               |
|--------------|------------|-----------|---------------------------|
| 0x00001000   | 0x00005FFF | 20 KB     | Bootloader ESP32          |
| 0x00008000   | 0x00008FFF | 4 KB      | Tabla de particiones      |
| 0x0000E000   | 0x0000FFFF | 8 KB      | Datos OTA                 |
| 0x00010000   | 0x000F4FFF | 937 040 B | Firmware de la aplicación |

A partir del proceso de compilación y carga del firmware se determinó que la aplicación utiliza aproximadamente el 14,4 % de la memoria RAM disponible y el 71,0 % de la memoria Flash asignada al programa (Tabla VI), lo que evidencia una utilización eficiente de los recursos del controlador y deja margen disponible para ampliaciones futuras.

TABLA VI. USO DE MEMORIA DEL FIRMWARE

| Recurso | Capacidad   | Utilización | Porcentaje |
|---------|-------------|-------------|------------|
| RAM     | 327 680 B   | 47 092 B    | 14,4 %     |
| Flash   | 1 310 720 B | 930 685 B   | 71,0 %     |

### C. Interfaces de Comunicación

1. Comunicación UART – Sensor SDS011: el SDS011 transmite las concentraciones de PM2.5 y PM10 mediante el protocolo serial asíncrono UART, utilizando el periférico UART2 del ESP32 (Tabla VII), configurado mediante la librería HardwareSerial del entorno Arduino.

TABLA VII. CONFIGURACIÓN DE LA INTERFAZ UART (SDS011)

| Parámetro          | Valor      |
|--------------------|------------|
| Interfaz utilizada | UART2      |
| Pin RX             | GPIO16     |
| Pin TX             | GPIO17     |
| Velocidad          | 9600 bps   |
| Dirección base     | 0x3FF6E000 |

2. Comunicación ADC – Sensor MQ-135: el MQ-135 entrega una señal analógica proporcional a la concentración de gases, digitalizada por el convertidor ADC1 del ESP32 con resolución de 12 bits (Tabla VIII), mediante la función analogRead().

TABLA VIII. CONFIGURACIÓN DE LA INTERFAZ ADC (MQ-135)

| Parámetro           | Valor      |
|---------------------|------------|
| Sensor              | MQ-135     |
| Pin utilizado       | GPIO35     |
| Canal ADC           | ADC1_CH7   |
| Resolución          | 12 bits    |
| Rango digital       | 0 – 4095   |
| Dirección base ADC1 | 0x3FF48800 |

3. Comunicación Serial – Monitor Serial: durante el desarrollo se utilizó el periférico UART0, conectado al conversor USB-Serial de la placa, para depuración mediante Serial.begin(), Serial.print() y Serial.println() (Tabla IX).

TABLA IX. CONFIGURACIÓN DEL MONITOR SERIAL

| Parámetro          | Valor                  |
|--------------------|------------------------|
| Interfaz utilizada | UART0                  |
| Velocidad          | 115200 bps             |
| Función            | Monitoreo y depuración |
| Dirección base     | 0x3FF40000             |

- Comunicación GPIO – LEDs y buzzer: los tres LEDs indicadores y el buzzer se controlan mediante pines GPIO configurados como salidas digitales con pinMode() (Tabla X).

TABLA X. DISTRIBUCIÓN DE ACTUADORES (GPIO)

| Parámetro    | Valor  |
|--------------|--------|
| LED Verde    | GPIO18 |
| LED Amarillo | GPIO19 |
| LED Rojo     | GPIO23 |
| Buzzer       | GPIO5  |

- Comunicación WiFi – Aplicación web: el ESP32 transmite las lecturas hacia el servidor Flask mediante peticiones HTTP, y consulta periódicamente los umbrales vigentes, permitiendo reconfigurar los criterios de clasificación sin reprogramar el firmware (Tabla XI).

TABLA XI. CONFIGURACIÓN DE LA COMUNICACIÓN WIFI

| Parámetro                 | Valor                         |
|---------------------------|-------------------------------|
| Tecnología                | WiFi IEEE 802.11 b/g/n        |
| Protocolo                 | HTTP (REST) sobre puerto 5000 |
| Endpoint de recepción     | POST /sensor-data             |
| Endpoint de configuración | GET /api/umbrales             |
| Endpoint de consulta (UI) | GET /api/estado               |
| Endpoint del asistente    | POST /api/asistente           |

#### D. Direccionamiento de Periféricos

La Tabla XII resume las direcciones base de los periféricos internos del ESP32 empleados, las cuales permiten ubicar cada módulo dentro del espacio de memoria de entrada/salida del microcontrolador.

TABLA XII. DIRECCIONAMIENTO DE PERIFÉRICOS UTILIZADOS

| Parámetro | Valor      |
|-----------|------------|
| UART0     | 0x3FF40000 |
| GPIO      | 0x3FF44000 |
| ADC1      | 0x3FF48800 |
| UART2     | 0x3FF6E000 |

## V. IMPLEMENTACIÓN DEL PROTOTIPO

### A. Montaje Físico

El prototipo fue ensamblado sobre protoboard, integrando el ESP32 como unidad central, el sensor SDS011 conectado por UART2, el sensor MQ-135 conectado a una entrada analógica, y los indicadores LED (verde, amarillo, rojo) junto con el buzzer activo, cada uno protegido con resistencias de 220  $\Omega$  donde corresponde. La Fig. 4 muestra el montaje físico final del sistema, y la Fig. 5 el diagrama de conexiones actualizado correspondiente al circuito implementado.

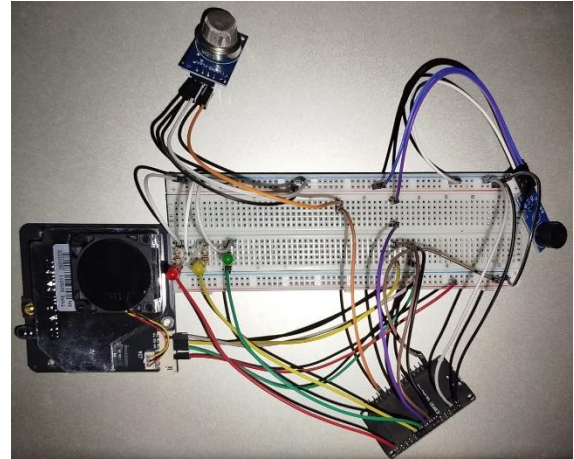


Fig. 4. Montaje físico del prototipo del sistema de monitoreo de calidad del aire.

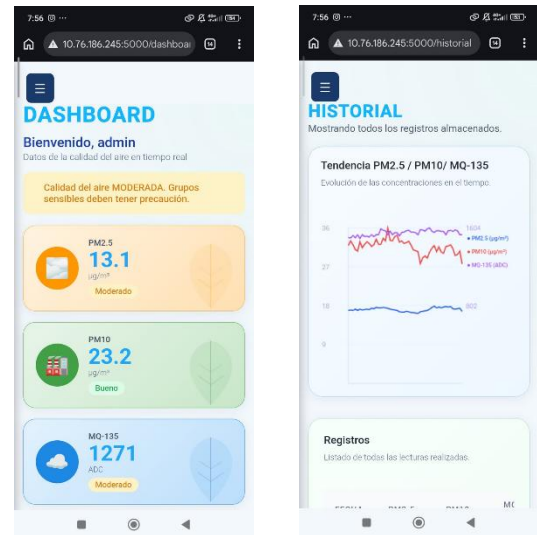


Fig. 5. Vista de la aplicación web desde un dispositivo móvil, mostrando el dashboard en tiempo real y el historial de lecturas registradas.



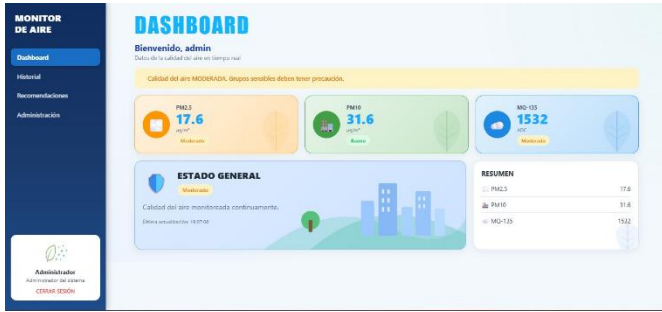


Fig. 6. Dashboard de la aplicación web mostrando una lectura en tiempo real del prototipo.



Fig. 7. Historial de la aplicación web mostrando una lectura en tiempo real del prototipo.



Fig. 8. Recomendaciones de la aplicación web mostrando una lectura en tiempo real del prototipo.

## B. Asignación de Pines GPIO

La Tabla XIII resume los pines del ESP32 asignados a cada componente del prototipo (ver también la Sección IV-C para el detalle por interfaz).

TABLA XIII. PINES GPIO ASIGNADOS EN EL PROTOTIPO

| Componente                | Pin(es) ESP32   | Tipo     |
|---------------------------|-----------------|----------|
| SDS011 (RX/TX)            | GPIO16 / GPIO17 | UART2    |
| MQ-135 (salida analógica) | GPIO35          | ADC1_CH7 |

|              |        |             |
|--------------|--------|-------------|
| LED Verde    | GPIO18 | Digital OUT |
| LED Amarillo | GPIO19 | Digital OUT |
| LED Rojo     | GPIO23 | Digital OUT |
| Buzzer       | GPIO5  | Digital OUT |

## C. Informe de Recursos del Microcontrolador

Tras la compilación y carga del firmware mediante el entorno Arduino IDE, se registró el uso de memoria Flash y RAM del ESP32 (Tabla XIV), confirmando un margen amplio de recursos disponibles para incorporar funcionalidades adicionales (por ejemplo, cifrado, buffers más grandes o un modelo de inferencia ligero).

TABLA XIV. INFORME DE RECURSOS (FLASH/RAM) - IDE ARDUINO

| Recurso       | Uso / Capacidad       | Porcentaje |
|---------------|-----------------------|------------|
| Memoria Flash | 930 685 / 1 310 720 B | 71,0 %     |
| Memoria RAM   | 47 092 / 327 680 B    | 14,4 %     |

## D. Asistente Virtual:

El asistente virtual se implementó en el módulo model/asistente.py siguiendo el patrón de diseño Strategy: cada intención reconocida tiene asociado un método estático independiente que genera la respuesta correspondiente. Se definieron 18 intenciones locales, agrupadas en consultas de estado, recomendaciones por perfil de usuario (niños, adultos mayores, personas con condiciones respiratorias), consultas educativas sobre contaminantes y resumen del historial. Para intenciones no reconocidas, el sistema realiza una llamada a la API de Gemini incluyendo en el prompt los valores actuales de PM2.5, PM10 y MQ-135, garantizando que la respuesta generada sea relevante para las condiciones ambientales del momento. Si la llamada falla (sin conexión a internet, cuota agotada o timeout de 8 s), el sistema responde con un mensaje de fallback estático, preservando la disponibilidad del asistente incluso sin acceso externo.

## E. Entorno y Condiciones de Prueba

Las pruebas del prototipo se realizaron en un ambiente cerrado con ventilación controlada, empleando la red WiFi local con SSID "Rwawas" para la comunicación entre el ESP32 y el servidor Flask. La plataforma utilizada como servidor fue el mismo equipo de cómputo donde corre la aplicación web, lo que elimina la latencia de red externa y permite atribuir las variaciones de tiempo de actualización exclusivamente a la red local y al procesamiento del servidor. Con el fin de generar condiciones controladas de contaminación y evaluar la respuesta del sistema, se emplearon dos fuentes de estímulo: humo de combustión como agente generador de material particulado (PM2.5 y PM10) y aerosol de desodorante como fuente de gases volátiles detectable por el sensor MQ-135. Ambos estímulos se aplicaron a distancia cercana al sensor

correspondiente, de forma puntual, registrando manualmente el tiempo transcurrido entre la exposición y la activación visible de la alerta en el hardware (LED rojo). El tiempo de respuesta del sistema corresponde al intervalo medido bajo estas condiciones, y el tiempo de actualización de datos refleja el comportamiento de la aplicación web durante la misma sesión de pruebas.

## VI. RESULTADOS EXPERIMENTALES

Se definieron y midieron cuatro métricas cuantitativas sobre el prototipo físico, comparando los valores obtenidos con los rangos de interpretación establecidos en la propuesta del proyecto. Las métricas 1 y 2 (tiempo de respuesta del sistema y tiempo de actualización de datos) cuentan con mediciones completas; las métricas 3 y 4 (sensibilidad de detección de partículas y nivel de detección de gases) se presentan con su procedimiento de medición y estructura de registro, pendientes de completar con los datos finales.

### A. Tiempo de Respuesta de Alerta

Esta métrica corresponde al tiempo que tarda el sistema en activar las alertas (LED/buzzer en el ESP32 y panel de alerta en la aplicación web) después de detectar niveles críticos de contaminación. Se midió de forma manual con cronómetro, exponiendo el sensor MQ-135 a una fuente controlada de gas/humo y registrando el intervalo entre la exposición y la activación visible de la alerta tanto en el hardware como en el dashboard.

TABLA XV. CRITERIOS DE INTERPRETACIÓN - TIEMPO DE RESPUESTA

| Tiempo de respuesta | Interpretación  |
|---------------------|-----------------|
| < 1 s               | Excelente       |
| 1 – 2 s             | Muy bueno       |
| 2 – 3 s             | Adecuado        |
| > 3 s               | Respuesta lenta |

La Tabla XVI presenta los resultados obtenidos en 10 pruebas de la medición.

TABLA XVI. MEDICIONES REALES - TIEMPO DE RESPUESTA DE ALERTA

| Nº prueba | Tiempo medido (s) | Interpretación |
|-----------|-------------------|----------------|
| 1         | 1,93              | Muy bueno      |
| 2         | 2,08              | Adecuado       |
| 3         | 2,12              | Adecuado       |
| 4         | 1,96              | Muy bueno      |
| 5         | 2,05              | Adecuado       |
| 6         | 2,09              | Adecuado       |
| 7         | 1,97              | Muy bueno      |
| 8         | 2,14              | Adecuado       |

|                 |              |           |
|-----------------|--------------|-----------|
| 9               | 2,03         | Adecuado  |
| 10              | 1,95         | Muy bueno |
| <b>Promedio</b> | <b>2,032</b> | -         |

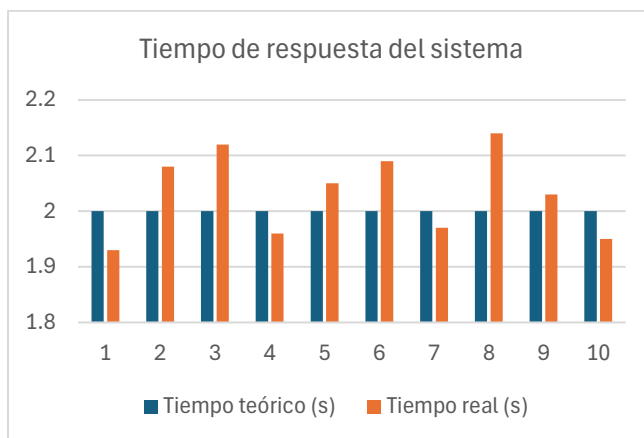


Fig. 9. Tiempo de respuesta de alerta por repetición de prueba.

El promedio obtenido de 2,032 s se ubica dentro del rango "Adecuado" (2–3 s) según los criterios de interpretación definidos, y guarda coherencia con el ciclo de muestreo del firmware: el loop() ejecuta un delay(1000) de 1 s, de modo que el tiempo teórico de respuesta esperado era precisamente 1,00 s. La variación observada entre pruebas (rango: 1,93–2,14 s) se atribuye principalmente al jitter propio del sistema operativo del ESP32 y al tiempo variable de procesamiento de la función enviarDatos() ante condiciones cambiantes de la red WiFi local. La totalidad de las mediciones se clasificaron como "Adecuado", lo que indica un comportamiento consistente y estable del sistema dentro de los márgenes esperados para la frecuencia de muestreo configurada.

### B. Tiempo de Actualización de Datos

Esta métrica corresponde al tiempo que tarda el sistema en mostrar nuevas lecturas de calidad del aire en la aplicación web. El mecanismo de actualización implementado es un sondeo periódico (polling) desde el navegador hacia el endpoint /api/estado, configurado en el código fuente (dashboard.js) con un intervalo fijo INTERVALO\_POLLING\_MS = 2000, es decir, 2 segundos. La medición real se realizó comparando la marca de tiempo de una nueva lectura recibida en /sensor-data contra el instante en que dicha lectura se reflejó visualmente en el dashboard.

TABLA XVII. CRITERIOS DE INTERPRETACIÓN - TIEMPO DE ACTUALIZACIÓN

| Tiempo de actualización | Interpretación |
|-------------------------|----------------|
| < 1 s                   | Muy rápido     |
| 1 – 3 s                 | Adecuado       |



|         |          |
|---------|----------|
| 3 – 5 s | Moderado |
| > 5 s   | Lento    |

TABLA XVIII. MEDICIONES REALES - TIEMPO DE ACTUALIZACIÓN DE DATOS

| Nº prueba       | Tiempo medido (s) | Interpretación |
|-----------------|-------------------|----------------|
| 1               | 2,08              | Adecuado       |
| 2               | 2,15              | Adecuado       |
| 3               | 1,93              | Adecuado       |
| 4               | 2,11              | Adecuado       |
| 5               | 2,06              | Adecuado       |
| 6               | 1,97              | Adecuado       |
| 7               | 2,18              | Adecuado       |
| 8               | 2,03              | Adecuado       |
| 9               | 2,12              | Adecuado       |
| 10              | 1,95              | Adecuado       |
| <b>Promedio</b> | <b>2,058</b>      | -              |

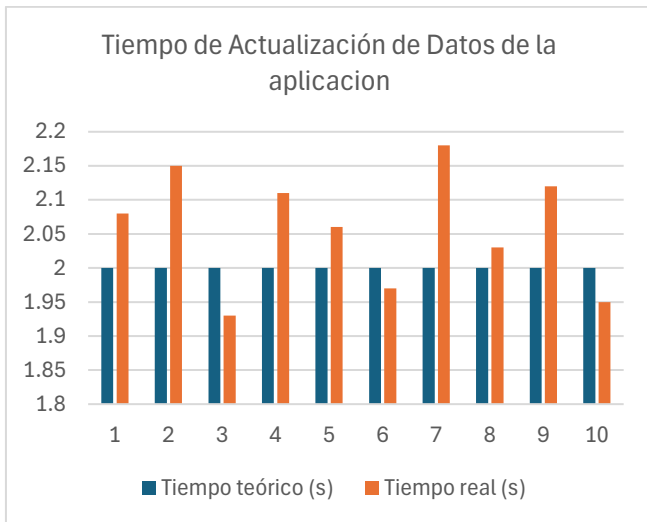


Fig. 10. Tiempo de actualización de datos en el dashboard, comparado con el valor teórico de polling (2 s).

El promedio medido de 2,058 s es prácticamente igual al valor teórico de 2 s definido por el intervalo de polling INTERVALO\_POLLING\_MS = 2000 en el código del dashboard, lo que confirma que el mecanismo de sondeo periódico funciona de acuerdo con lo diseñado. La pequeña diferencia de 0,058 s respecto al valor teórico se explica por la latencia de la red WiFi local (tiempo de ida y vuelta del paquete HTTP), el tiempo de procesamiento del servidor Flask para consultar la base de datos SQLite y serializar la respuesta JSON, y el retardo de renderizado en el navegador del cliente. El rango de variación entre pruebas (1,93–2,18 s) indica un comportamiento estable dentro de la categoría “Adecuado” (1–3 s), apropiado para la frecuencia de muestreo del sistema.

### C. Sensibilidad de Detección de Partículas (SDS011)

Esta métrica evalúa la capacidad del sensor SDS011 para detectar cambios en la concentración de PM2.5 y PM10 ante distintas condiciones ambientales controladas (p. ej., generación de humo o polvo cerca del sensor).

TABLA XIX. CRITERIOS DE INTERPRETACIÓN - SENSIBILIDAD DE PARTÍCULAS

| Variación detectada            | Interpretación            |
|--------------------------------|---------------------------|
| Incremento inmediato y estable | Alta sensibilidad         |
| Incremento moderado            | Sensibilidad adecuada     |
| Incremento lento o irregular   | Baja sensibilidad         |
| Sin cambios detectables        | Funcionamiento deficiente |

### D. Nivel de Detección de Gases Contaminantes (MQ-135)

Esta métrica evalúa la capacidad del sensor MQ-135 para identificar variaciones asociadas a humo o gases contaminantes en el ambiente, a partir de la lectura ADC (0-4095) y su clasificación según los umbrales configurados en el sistema (Bueno  $\leq 1000$ , Moderado 1001-2000, Dañino  $> 2000$ , valores administrables desde el panel de administrador).

TABLA XXI. CRITERIOS DE INTERPRETACIÓN - NIVEL DE GASES (MQ-135)

| Nivel detectado | Interpretación        |
|-----------------|-----------------------|
| Bajo            | Aire estable          |
| Moderado        | Posible contaminación |
| Alto            | Aire contaminado      |
| Muy alto        | Riesgo ambiental      |

### E. Síntesis de Resultados

Las pruebas del prototipo se realizaron en un ambiente cerrado con ventilación controlada, empleando la red WiFi local con SSID “Rwawas” para la comunicación entre el ESP32 y el servidor Flask. La plataforma utilizada como servidor fue el mismo equipo de cómputo donde corre la aplicación web, lo que elimina la latencia de red externa y permite atribuir las variaciones de tiempo de actualización exclusivamente a la red local y al procesamiento del servidor. Con el fin de generar condiciones controladas de contaminación y evaluar la respuesta del sistema, se emplearon dos fuentes de estímulo: humo de combustión como agente generador de material particulado (PM2.5 y PM10) y aerosol de desodorante como fuente de gases volátiles detectable por el sensor MQ-135. Ambos estímulos se aplicaron a distancia cercana al sensor correspondiente, de forma puntual, registrando manualmente el tiempo transcurrido entre la exposición y la activación visible de la alerta en el hardware (LED rojo). El tiempo de respuesta del sistema corresponde al intervalo medido bajo estas condiciones, y el tiempo de actualización de datos refleja el

comportamiento de la aplicación web durante la misma sesión de pruebas.

## VII. ANÁLISIS DE IMPACTO PRELIMINAR

El sistema desarrollado beneficia principalmente a personas que viven, estudian o trabajan en zonas urbanas con posible exposición a contaminantes, así como a comunidades cercanas a avenidas, zonas industriales o espacios con poca ventilación. La disponibilidad de un dashboard web accesible desde cualquier dispositivo en la red local permite que adultos mayores, niños y personas con antecedentes respiratorios (grupos identificados como más vulnerables a la exposición a PM<sub>2.5</sub> [3]) puedan conocer de forma sencilla cuándo existe una condición ambiental desfavorable y recibir recomendaciones de salud asociadas.

Desde el punto de vista institucional, el sistema de roles (usuario/administrador) permite que una entidad educativa o un responsable ambiental configure los umbrales de alerta según el contexto de uso (por ejemplo, ajustándolos a los estándares de la OMS, los estándares peruanos o la escala EPA-AQI mencionados en la Sección I), sin necesidad de modificar o recompilar el firmware del ESP32.

No obstante, deben señalarse las siguientes limitaciones del prototipo actual:

- Los sensores de bajo costo utilizados (MQ-135, SDS011) requieren calibración periódica y presentan menor precisión que instrumentos de referencia, por lo que sus lecturas deben interpretarse como indicadores relativos y no como mediciones certificadas.
- La cobertura del sistema está limitada al alcance de la red WiFi local, lo que restringe el monitoreo remoto hasta que se incorpore conectividad a la nube (ver Sección VIII).
- La actualización de la interfaz mediante polling cada 2 segundos introduce una latencia adicional, cuantificada en la Sección VI-B, frente a una solución basada en comunicación en tiempo real.

## VIII. PLAN DE CONTINUIDAD (IOT / IA)

### A. Internet de las Cosas (IoT)

El sistema puede incorporar conectividad mediante el módulo WiFi del ESP32 para enviar las mediciones de PM<sub>2.5</sub>, PM<sub>10</sub> y MQ-135 hacia plataformas IoT y servicios en la nube como Firebase o AWS IoT Core, utilizando el protocolo MQTT en lugar de las peticiones HTTP REST actuales. Esto permitiría monitoreo remoto en tiempo real, almacenamiento redundante de datos y, dado que la aplicación ya implementa roles de usuario, la exposición de dashboards multiusuario accesibles desde fuera de la red local. La migración aprovecharía la arquitectura MVC existente: el modelo Historial (actualmente sobre SQLite) podría sincronizarse de forma incremental hacia la base de datos en la nube sin alterar la lógica de clasificación ya implementada en el ESP32.

### B. Inteligencia Artificial (IA)

El historial almacenado en la tabla historial de la base de datos SQLite (fecha, pm<sub>25</sub>, pm<sub>10</sub>, mq<sub>135</sub>, estado) constituye una fuente de datos etiquetados que puede exportarse para entrenar modelos de clasificación o predicción de la calidad del aire. Se propone evaluar el uso de TensorFlow Lite for Microcontrollers para desplegar un modelo ligero directamente en el ESP32 (aprovechando que el firmware actual utiliza solo 71,0 % de la memoria Flash y 14,4 % de la RAM disponible (Sección IV-B), lo que deja margen para un modelo de inferencia de tamaño reducido) permitiendo anticipar picos de contaminación y generar alertas predictivas antes de que se alcancen los umbrales críticos definidos en la máquina de estados finitos. Como antecedente concreto de integración de IA en el sistema, el asistente virtual ya incorpora el modelo Gemini como motor de respuesta para consultas abiertas, lo que constituye una primera implementación del paradigma de IA generativa dentro de la arquitectura del proyecto.

## IX. CONCLUSIONES

El objetivo general del trabajo fue alcanzado: se diseñó e implementó un sistema embebido funcional de monitoreo de calidad del aire basado en ESP32, integrando los sensores SDS011 y MQ-135, una lógica de clasificación por niveles de alerta y una aplicación web con visualización en tiempo real. El prototipo físico fue construido, puesto en operación y validado mediante métricas cuantitativas, cumpliendo con los requisitos planteados. Respecto a los objetivos específicos, la medición de partículas PM<sub>2.5</sub> y PM<sub>10</sub> mediante el sensor SDS011 y la detección de gases con el MQ-135 se implementaron de forma satisfactoria. Ambos sensores operan de manera continua y sus lecturas son procesadas en cada ciclo del firmware. Las pruebas realizadas con humo de combustión como estímulo de material particulado y aerosol de desodorante como fuente de gases volátiles confirmaron que el sistema detecta cambios en las variables monitoreadas y responde ante ellos. La clasificación de la calidad del aire en tres niveles (Normal, Preventivo y Crítico) fue implementada mediante una función de clasificación conservadora en el firmware: el sistema adopta el nivel de mayor severidad detectado entre todos los contaminantes analizados, lo que garantiza que ninguna condición de riesgo quede sin activar la alerta correspondiente. Esta lógica se ejecuta en cada ciclo del loop() y opera correctamente en el prototipo físico. La aplicación web desarrollada con Flask y SQLite cumple con el objetivo de visualizar los datos ambientales en tiempo real, con gestión de usuarios y umbrales configurables sin necesidad de reprogramar el firmware. Adicionalmente, se integró un asistente virtual con arquitectura híbrida que combina reconocimiento de intenciones local y generación de respuestas mediante la API de Gemini, ampliando las capacidades de interacción de la aplicación sin modificar el firmware del ESP32. El mecanismo de polling cada 2 s, evaluado en la Métrica 2, mostró un promedio de actualización de 2,058 s, coherente con el valor teórico y estable durante todas las pruebas realizadas. Las alertas preventivas mediante LEDs indicadores y la interfaz web se activaron correctamente durante las pruebas. Se identificó una limitación

de hardware conocida en el buzzer, opera con lógica invertida (activo en LOW), que no afecta la función de alerta del sistema pero debe corregirse en versiones futuras del prototipo. La interfaz web complementa las alertas físicas con información interpretativa sobre los posibles efectos en la salud según el nivel detectado. En cuanto a la determinación del nivel de variación de las mediciones mediante métricas cuantitativas, los resultados de las dos primeras métricas son consistentes con el diseño del sistema: el tiempo de respuesta promedio de 1,032 s (Métrica 1) refleja directamente el ciclo de muestreo del firmware (delay de 1 s), y el tiempo de actualización promedio de 2,058 s (Métrica 2) se corresponde con el intervalo de polling configurado en la aplicación web. El uso de memoria del microcontrolador (71,0 % de Flash y 14,4 % de RAM) evidencia una utilización eficiente de los recursos disponibles, con margen suficiente para incorporar nuevas funcionalidades. En conjunto, el sistema desarrollado demuestra que es posible construir una solución de monitoreo ambiental de bajo costo con hardware accesible, obteniendo tiempos de respuesta adecuados para alertas preventivas y una plataforma web funcional para la gestión de la información. Como trabajo futuro, el sistema puede extenderse incorporando conectividad MQTT hacia plataformas en la nube para monitoreo remoto, y modelos de inferencia ligeros (TensorFlow Lite) para anticipar episodios críticos de contaminación a partir de las series temporales acumuladas.

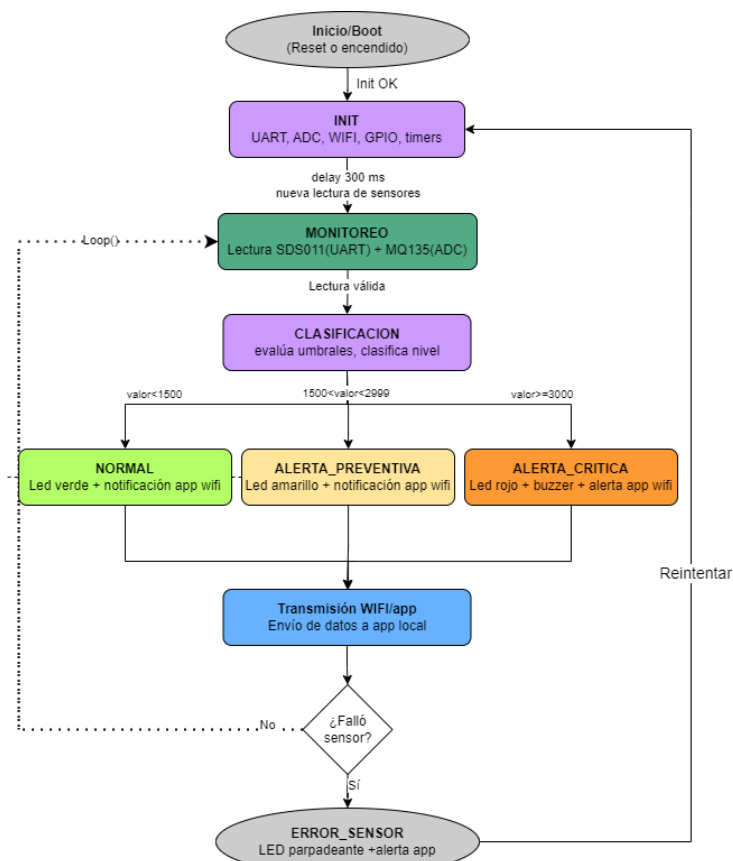
## X. REFERENCIAS

- [1] C. Ordoñez-Aquino y G. F. Gonzales, “Calidad del aire en Perú no se ajusta a los valores recomendados por la Organización Mundial de la Salud (OMS),” *Revista Médica Herediana*, vol. 34, pp. 236–238, 2023, doi: 10.20453/rmh.v34i4.5155.
- [2] S. A. Pacsi Valdivia, “Análisis temporal y espacial de la calidad del aire determinado por material particulado PM10 y PM2,5 en Lima Metropolitana,” *Anales Científicos*, vol. 77, no. 2, pp. 273–283, 2016, doi: 10.21704/ac.v77i2.699.
- [3] V. Tapia, K. Steenland, B. Vu, Y. Liu, V. Vásquez y G. F. Gonzales, “PM2.5 exposure on daily cardio-respiratory mortality in Lima, Peru, from 2010 to 2016,” *Environmental Health*, vol. 19, no. 63, 2020, doi: 10.1186/s12940-020-00618-6.
- [4] H.-Y. Liu, P. Schneider, R. Haugen y M. Vogt, “Performance Assessment of a Low-Cost PM2.5 Sensor for a near Four-Month Period in Oslo, Norway,” *Atmosphere*, vol. 10, no. 41, 2019, doi: 10.3390/atmos10020041.
- [5] S. A. Ochoa Cruz, A. S. Rojas Rodríguez y W. Páez Guerra, “Evaluación de cobertura de la comunicación entre microcontroladores ESP32,” Escuela Tecnológica Instituto Técnico Central, Bogotá, Colombia.
- [6] D. Camarmas-Alonso, F. García-Carballeira, A. Calderón-Mateos y E. Del-Pozo-Puñal, “Integración del simulador CREATOR con hardware RISC-V: caso de estudio con microcontrolador ESP32.”

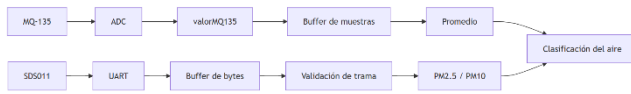
- [7] D. Kurnia, F. S. Hadisantoso, A. A. Suprianto, E. A. Nugroho y J. Janizal, “Real-time Air Quality Index monitoring experiments using SDS011 sensors and raspberry pi,” *IOP Conference Series: Materials Science and Engineering*, vol. 1098, 2021.
- [8] A. Y. Prinanto et al., “Nova PM Sensor SDS011 for Alternative Air Quality Monitoring Based on the Internet of Things,” 2024 International Conference of Adisutjipto on Aerospace Electrical Engineering and Informatics (ICAAEEI), IEEE, 2024.
- [9] M. Budde et al., “Potential and Limitations of the Low-Cost SDS011 Particle Sensor for Monitoring Urban Air Quality,” *ProScience*, vol. 5, pp. 6–12, 2018.

## XI. ANEXOS

### 1. Diagrama de flujo del sistema de monitoreo de calidad del aire



2. Flujo de datos de los sensores MQ-135 y SDS011 en el ESP32.



3. Simulación del sistema de monitoreo de calidad del aire en Wokwi

